

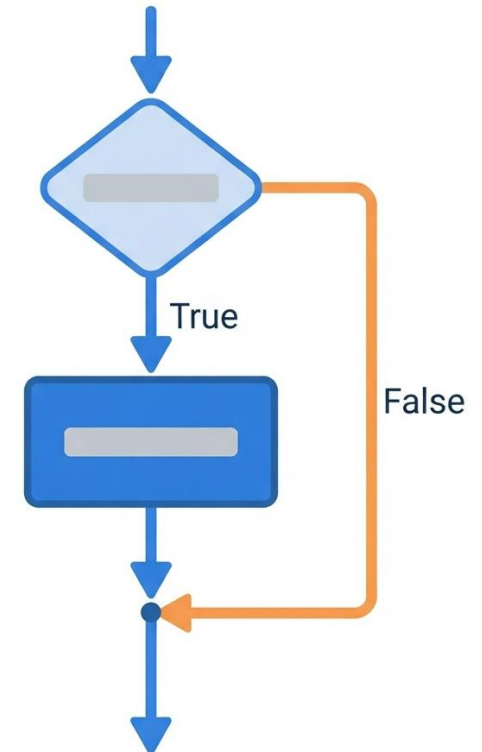
Проекти з розгалуженнями. Коротка форма

Тема 3. Алгоритми та програми · § 4.6, с. 133–136

У Python межі блоку задає ВІДСТУП

Якщо так — зроби. Якщо ні — нічого.

- Кожне розгалуження починається з команди визначення значення логічного виразу.
- Результат — тільки True або False. Третього не буває.
- True → виконуються команди, далі — наступна команда після розгалуження.
- False → ОДРАЗУ наступна команда після розгалуження. Нічого не сталося.
- ★ У короткій формі немає варіанта «інакше». Він з'явиться на уроці 20.
- Умовами були висловлювання, рівності, нерівності. Усі вони — логічні вирази.



if — і двокрапка

- if <логічний вираз>:
- <послідовність команд>
- Читається просто: ЯКЩО вираз істинний — зроби це.
- ⚠ Двокрапка обов'язкова. Забули → `SyntaxError: expected ':'`
- Якщо команда ОДНА — можна в тому самому рядку: `if x < 0: x = -x`
- Це прямо дозволено підручником, с. 136.

Шість операцій — і один знак, якого тут немає

- == дорівнює != не дорівнює
- > більше < менше
- >= більше або дорівнює <= менше або дорівнює
- ⚠ Знака = у цій таблиці НЕМАЄ. = це не порівняння.
- ★ = означає ПРИСВОЇТИ. == означає ПОРІВНЯТИ. У розгалуженні завжди ==
- Написали if x = 0: → SyntaxError: ... Maybe you meant '==' or ':=' instead of '='?

Марійка нарешті скаже словами

- На практичній проєкт показував -68 і не міг сказати «не вистачило».
- Додаємо до моделі: якщо $r < 0 \rightarrow$ повідомити «Не вистачило грошей!»
- `if r < 0:`
- `showinfo('Увага', 'Не вистачило грошей!')`
- `label5['bg'] = 'red'`
-  Рядок із результатом лишається БЕЗ відступу — він працює завжди.

Відступ вирішує, що всередині розгалуження

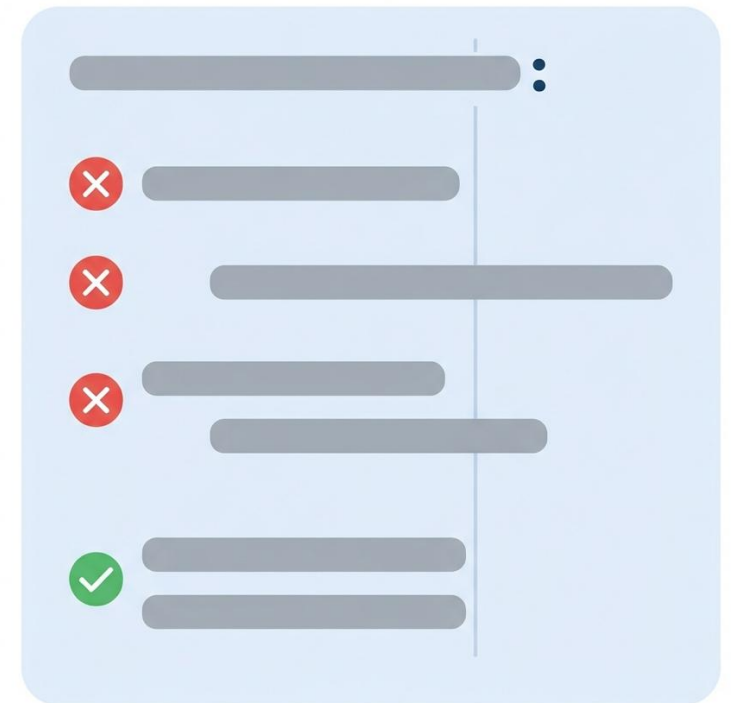
- `if x < 0:`
- `a = 1` ← ЦЕ у розгалуженні
- `b = 2` ← І ЦЕ теж
- `c = 3` ← А ЦЕ виконається ЗАВЖДИ
- ★ У Python межі блоку задає ВІДСТУП. Не дужки, не `begin/end`.
- ⚠ Посунули рядок вправо — проєкт працює в ОДНОМУ випадку з ДВОХ. І помилки немає.

[Місце для ілюстрації]

Аркуш коду з п'яти смужок: перша з двокрапкою біля лівого краю, дві зсунуті вправо в оранжевій пунктирній рамці з фігурною дужкою, порожній рядок, п'ята знову біля краю з іконкою нескінченності. Мірна стрілка з чотирма засічками показує відступ

П'ять повідомлень, які ти побачиш сьогодні

- IndentationError: expected an indented block after 'if' statement → тіло БЕЗ відступу
- IndentationError: unexpected indent → відступ зайвий або відступи в блоці різні
- TabError: inconsistent use of tabs and spaces → змішали Tab і пробіли
- ★ TabError найпідступніша: на екрані ВСЕ ВИГЛЯДАЄ РІВНО. Екран бреше.
- Лікується однією командою: виділити код → Format ⇒ Untabify Region
- SyntaxError ... Maybe you meant '==' → Python сам написав відповідь. ЧИТАЙТЕ повідомлення.



Модуль числа — найкоротше розгалуження

- `x = float(entry.get())`
- `if x < 0: x = -x`
- `result['text'] = 'Модуль: ' + str(x)`
- ⚠ `abs()` не використовувати — сенс завдання саме в розгалуженні.
- Скільком значень перевірити? ★ ТРИ: -7, 7 і НУЛЬ.
- Нуль — це МЕЖА. Правило з уроку 13: тестуй межу, а не середину.

Домашнє завдання

- 1. § 4.6, с. 133–136. Блок-схему НЕПОВНОГО розгалуження — у зошит, підписати True і False.
- 2. Доробити Марійку → `marijka_if.py`. ⚠ ДВА скріншоти: один із повідомленням, один БЕЗ.
- 3. «Модуль числа» → `modul.py`, без `abs()`. Перевірити -7, 7 і 0.
- 4. (за бажанням) Захист від порожнього поля: `if entry.get() == " ... return`
- Приблизно 25 хвилин. Строк — до 02.12.
- ⚠ `IndentationError` або `TabError` — це ВІДСТУПИ, а не логіка. Не шукайте помилку у формулі.